

Contents

Präsentation Framework-Walkthrough — konkrete Regieanweisung	1
Die Grundentscheidung: Live-Code statt fertiger Folien	1
Vor dem Block: Technik-Checkliste	1
Folie 1 — Einstieg (die einzige Folie vor dem Live-Teil)	2
Schritt 1 — Live-Demo (5 Min)	2
Schritt 2 — Grundkonzepte erklären (10 Min, optional kürzbar)	2
Schritt 3 — Editor: Models.kt gemeinsam lesen (25 Min)	2
Schritt 4 — Tafel: wie ruft die Engine decide() auf? (10 Min)	3
Schritt 5 — Editor: RandomBot.kt Zeile für Zeile (20 Min)	3
Schritt 6 — Editor: eigene Team-Datei live bearbeiten (15 Min)	3
Folie 2 — Übergang zu Arbeitsblock 2	4
Zeitpuffer	4

Präsentation Framework-Walkthrough — konkrete Regieanweisung

Diese Datei beantwortet die Frage *“Wie präsentiere ich das eigentlich?”* für den Block aus [tag-1-framework-walkthrough.md](#). Sie ist bewusst als Schritt-für-Schritt-Drehbuch geschrieben, das man während des Blocks neben sich liegen haben kann — mit Sprechbeispielen, nicht nur Stichworten.

Die Grundentscheidung: Live-Code statt fertiger Folien

Für diesen Block werden **keine klassischen Präsentationsfolien** (PowerPoint o.ä.) für den Code-Teil empfohlen. Stattdessen: **Beamer zeigt abwechselnd Editor, Terminal und die laufende App**. Nur für Anfang und Übergang gibt es je eine kurze Folie (siehe unten).

Warum kein Folien-Foliensatz für den Code? - Schüler sehen echten, klickbaren Code — keine Screenshots, die schon beim Erstellen veraltet sein können. - Wenn eine Frage kommt (“was macht `.random()` nochmal genau?”), kann man direkt im Editor nachschauen oder ausprobieren, statt auf eine feste Folienreihenfolge angewiesen zu sein. - Der Kontrast “das hier ist fertiges Framework” vs. “das hier schreibt ihr selbst” wirkt am stärksten, wenn beide Dateien als Tabs im selben Editor-Fenster offen sind und man einfach zwischen ihnen hin- und herklickt.

Was du technisch brauchst: 1. Terminal-Fenster (für `./gradlew run`) 2. Editor (IntelliJ oder VS Code) mit bereits geöffnetem Projekt 3. Beide Fenster nebeneinander oder schnell per Tastenkombination wechselbar

Vor dem Block: Technik-Checkliste

Unbedingt **vor** den Schülern durchgehen, nicht live improvisieren:

- ☐ Projekt einmal lokal bauen (`./gradlew build`), damit die Live-Demo nicht an einem kalten Gradle-Cache/Download hängen bleibt
 - ☐ Editor-Schriftgröße hochstellen (mindestens 18pt, besser 20pt) — sonst ist von hinten im Raum nichts lesbar
 - ☐ Terminal-Schriftgröße ebenfalls hochstellen
 - ☐ Beamer-Auflösung/Spiegelung testen, bevor Schüler im Raum sind
 - ☐ Diese Dateien als Tabs vorab öffnen, in dieser Reihenfolge: `Models.kt`, `RandomBot.kt`, `TeamABots.kt` (bzw. die für die anwesenden Teams relevante(n) Team-Datei(en))
 - ☐ `docs/scrum-board.md` und ausgedruckte Backlog-Karten (siehe [pdf/README.md](#)) am Board bereitlegen für den Übergang danach
-

Folie 1 — Einstieg (die einzige Folie vor dem Live-Teil)

Eine einzelne Folie oder alternativ ein Whiteboard-Anschrieb, bevor irgendetwas gestartet wird:

Titel: Framework-Walkthrough — was baut ihr heute?

Inhalt (2-3 Stichpunkte, groß geschrieben): - Sprint-Ziel: *“Bot mit mindestens zwei Verhaltensregeln, der ein Testduell übersteht.”* - Ein Satz, den man auch laut sagen sollte: *“Ihr schreibt heute eine einzige Funktion. Alles andere — Spielfeld, Regeln, Anzeige — ist schon fertig.”*

Danach die Folie weglegen/ausblenden und direkt zu Schritt 1 wechseln.

Schritt 1 — Live-Demo (5 Min)

Wechsel zu: Terminal, dann App-Fenster.

1. Terminal einblenden, `./gradlew run` eintippen und ausführen (dauert je nach Rechner 10-30 Sekunden — diese Zeit für den Satz von Folie 1 nutzen, nicht schweigend warten).
2. Sobald die App startet: RandomBot und ChaserBot als Teilnehmer auswählen, Match starten.
3. App-Fenster maximieren, damit alle im Raum das Raster sehen.

Sprechtext während das Match läuft: *“Schaut euch das an — zwei fertige Bots, die gegeneinander kämpfen. Das ist keine Zauberei, das ist Code, den wir uns jetzt gemeinsam anschauen. Am Ende des heutigen Tages soll euer eigener Bot hier genauso mitspielen können.”*

Kurz auf UI-Elemente zeigen (nicht erklären, nur benennen): *“Links das Spielfeld, rechts Scoreboard und Log — da seht ihr später, was euer Bot in jedem Moment tut.”*

Schritt 2 — Grundkonzepte erklären (10 Min, optional kürzbar)

Wechsel zu: nichts — Tafel/Whiteboard oder einfach im Stehen erklären, noch kein Code zeigen.

Nur falls die Konzepte Interface/Enum/data class aus dem Kotlin-Vorlauf noch nicht sicher sitzen (siehe Abschnitt 0 in tag-1-framework-walkthrough.md für die genauen Erklärtexte und Analogien). Wenn die Klasse das schon kann: diesen Schritt weglassen und die Zeit in Schritt 5 investieren.

Kurzform der vier Analogien, die man parat haben sollte: - Interface = Stellenausschreibung (“kann kochen”, aber nicht wie) - Enum = Ampel (nur Rot/Gelb/Grün, nichts dazwischen) - data class = Karteikarte mit automatischem “sind zwei Karten gleich?”-Check - sealed interface = “genau eine von mehreren fest definierten Möglichkeiten”

Schritt 3 — Editor: Models.kt gemeinsam lesen (25 Min)

Wechsel zu: Editor, Tab Models.kt.

Von oben nach unten scrollen, bei jedem Typ kurz stehen bleiben. Reihenfolge exakt wie im Walkthrough-Dokument: Direction -> Position -&; RobotState -> Sensors -> Action -> RobotBrain.

Konkrete Methode, die Aufmerksamkeit hochhält: Bei jedem neuen Typ erst laut die Frage stellen *“Was glaubt ihr, wofür ist das da?”*, 2-3 Antworten aus dem Raum sammeln, dann erst die eigene Erklärung geben (Texte dazu stehen in tag-1-framework-walkthrough.md, Abschnitt 2).

Bei jedem Typ zusätzlich ein Nutzungsbeispiel zeigen, nicht nur die Definition. Das Walkthrough-Dokument enthält zu Position, RobotState, Sensors und Action jeweils 1-2

fertige Codeschnipsel aus Schülersicht (z.B. `if (sensors.self.health < 20) { ... }`). Diese am besten direkt in einer Scratch-Datei oder einem Kommentar im Editor eintippen und laufen lassen (oder zumindest zeigen, wie sie aussehen würden) — die reine Typ-Definition allein bleibt abstrakt, das Beispiel macht den Bezug zur eigenen Aufgabe sichtbar.

Besonders hervorheben mit einem Beispiel an der Tafel: Bei `Position.moved()` ein konkretes Zahlenbeispiel durchrechnen (Roboter auf (3,3), Schritt NORTH, neue Position (3,2)) — abstrakte Formeln bleiben bei 10.-Klässlern schlechter hängen als ein durchgerechnetes Beispiel.

Bei Action besonders auf die Klammer-Falle hinweisen: `Action.Wait` ohne Klammern, `Action.Move(Direction.NORTH)` mit Klammern und Parameter — das ist ein Detail, das beim ersten eigenen Schreiben fast immer zu einem Compile-Fehler führt, wenn es nicht vorher explizit erwähnt wurde.

Schritt 4 — Tafel: wie ruft die Engine `decide()` auf? (10 Min)

Wechsel zu: weg vom Beamer, hin zur Tafel/zum Whiteboard. Das ist bewusst ein Medienwechsel — signalisiert “jetzt nicht mehr Kotlin-Syntax, jetzt der große Ablauf”.

Das Diagramm aus `tag-1-framework-walkthrough.md` (Abschnitt 3) an die Tafel malen:

Sensors bauen → `decide()` aufrufen → Bewegungen auflösen → Schaden anwenden

Danach die zwei kleinen Geschichten erzählen (Bewegungs-Konflikt, gesammelter Schaden — Wortlaut steht im Walkthrough-Dokument). Diese bewusst **als Geschichte erzählen**, nicht an die Tafel schreiben — es sind spätere Aha-Momente, keine Merksätze zum Auswendiglernen.

Schritt 5 — Editor: `RandomBot.kt` Zeile für Zeile (20 Min)

Wechsel zu: Editor, Tab `RandomBot.kt`.

Das ist der wichtigste Programmpunkt des ganzen Blocks — hier sehen Schüler zum ersten Mal, wie ein `RobotBrain` tatsächlich implementiert aussieht.

Konkrete Methode: Vor jeder Zeile fragen “*Was glaubt ihr, macht diese Zeile?*” und 10-15 Sekunden Stille aushalten, bevor man selbst erklärt — das ist unbequem, aber aktiviert deutlich mehr als reines Vorlesen.

Erklärungen zu jeder Zeile stehen in `tag-1-framework-walkthrough.md`, Abschnitt 5.

Schritt 6 — Editor: eigene Team-Datei live bearbeiten (15 Min)

Wechsel zu: Editor, Tab `TeamABots.kt` (bzw. passende Team-Datei).

1. Zeigen, wie die Datei aktuell aussieht (`Action.Move(Direction.SOUTH)` als Platzhalter).
2. **Live** ändern zu `Action.Move(Direction.entries.random())`.
3. Zurück zum Terminal wechseln, `./gradlew run` erneut starten.
4. Zeigen, dass der Bot sich jetzt zufällig bewegt.

Sprechtext: “*Das war’s schon — eine Zeile geändert, neu gestartet, der Bot verhält sich anders. Genau das macht ihr jetzt gleich selbst, nur mit eurer eigenen Logik.*”

Danach explizit den häufigsten Fehler ankündigen: “*Wenn ihr eine ganz neue Bot-Klasse anlegt: vergesst nicht, sie unten in die Liste einzutragen — sonst taucht sie nicht in der Bot-Auswahl auf. Das ist der Fehler, der euch als Erstes passieren wird, und das ist völlig normal.*”

Folie 2 — Übergang zu Arbeitsblock 2

Titel: Jetzt seid ihr dran

Inhalt: - Story 1.1 (Zufällige Bewegung) als Einstieg - Erinnerung: Pair-Programming, Driver/Navigator wechseln sich ab - Erinnerung: neue Klasse -> in teamXBots-Liste eintragen

Danach direkt zu den Rechnern schicken, Board zeigen (Karten liegen schon in der Backlog-Spalte, siehe scrum-board.md).

Zeitpuffer

Der Block ist mit 90 Minuten eng getaktet (siehe Zeitbudget-Tabelle in tag-1-framework-walkthrough.md). Wenn Zeit fehlt: zuerst Schritt 2 (Grundkonzepte) kürzen oder streichen, das lässt sich am ehesten in Schritt 3 nebenbei mit erklären. Schritt 5 (RandomBot.kt) und Schritt 6 (eigene Datei) sind die beiden Schritte, die am wenigsten gekürzt werden sollten — das sind die Momente, an denen Schüler den Bezug zu ihrer eigenen Aufgabe herstellen.